TESTE DE SOFTWARE

Trabalho Prático

Detecção de Bad Smells e Refatoração Segura

Aluno: Paulo Henrique Fonseca de Assis

Matrícula: 829040

Repositório: github.com/Paulooassis/bad-smells-js-refactoring

Novembro de 2026

1. Preparação do ambiente

Antes de qualquer análise, a rede de segurança precisava estar funcionando, e ela não estava. Na primeira execução de `npm test` a suíte nem chegou a rodar: o projeto declara `"type": "module"` e o Jest 29 só entende `import`/`export` nativos quando o Node é iniciado com `--experimental-vm-modules`. Ajustei o script de teste no `package.json` para isso.

Com os testes rodando, 4 de 5 passaram. O que falhou foi *"deve gerar um relatório HTML completo para Admin (com prioridade)"*. No ramo do administrador a linha era montada como `<tr ${style}>`; quando o item não é prioritário, `style` fica vazio e sobra `<tr >` com um espaço. Corrigi passando o espaço para dentro da própria string de estilo (duas linhas alteradas, nenhuma no teste). Só depois de ver os 5 testes passando comecei a refatorar. Cada um desses ajustes ficou em um commit separado no repositório.

2. Análise de smells

2.1 Método longo (Long Method)

O `generateReport` tem cerca de 60 linhas e faz sozinho cinco coisas: escreve o cabeçalho, decide quais itens o usuário pode ver, marca os itens prioritários, formata cada linha e escreve o rodapé com o total. Na manutenção, qualquer mudança pequena (uma coluna nova, uma regra de filtro diferente) obriga a entender o método inteiro, porque está tudo misturado no mesmo laço. Nos testes, não existe forma de testar a formatação HTML separada da regra de permissão: o único ponto de entrada é o método todo. Quando um teste falha, não diz qual das cinco partes quebrou.

2.2 Condicionais repetidas e aninhadas (Repeated Switches)

A pergunta "o relatório é CSV ou HTML?" é feita quatro vezes: no cabeçalho, duas vezes dentro do laço (no ramo ADMIN e no ramo USER) e no rodapé. O papel do usuário também é decidido com `if/else` dentro do laço, e no caminho mais fundo o código chega a quatro níveis de aninhamento (`for` → papel → valor → formato). Para incluir um formato novo, como JSON, seria preciso alterar quatro blocos e não esquecer nenhum; se esquecer um, o relatório sai pela metade sem erro algum. Para os testes, cada combinação papel × formato é um caminho diferente no código, e cada formato ou papel novo multiplica a quantidade de casos.

2.3 Código duplicado (Duplicated Code)

A linha CSV `${item.id},${item.name},${item.value},${user.name}` aparece duas vezes, a linha HTML aparece duas vezes (uma com estilo, outra sem) e `total += item.value` aparece em quatro ramos. O defeito do `<tr >` é consequência direta disso: as duas cópias da linha HTML evoluíram separadas e só uma delas ficou errada. Para os testes, isso significa que um teste passando para USER não garante nada sobre ADMIN, porque o código executado é outro.

Também anotei pontos menores: os números mágicos `500` e `1000` sem nome, as strings `'ADMIN'`, `'CSV'` etc. soltas no código, o método alterar os objetos recebidos (`item.priority = true`) e o atributo `this.db`, que nunca é usado.

3. Relatório da ferramenta

O repositório já trazia o `.eslintrc.json` e as dependências `eslint` (8.x) e `eslint-plugin-sonarjs` (0.21) no `package.json`, então bastou o `npm install`. Mantive a configuração do repositório, que é mais rígida que a do enunciado (limite de complexidade 5 em vez de 15). Testei também com 15 e o resultado foi o mesmo.

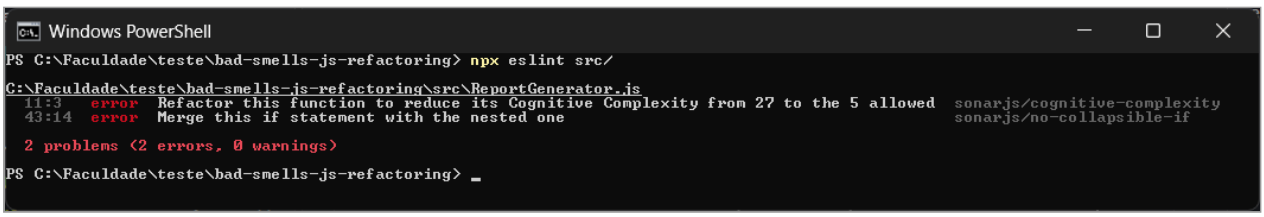


Figura 1: resultado de `npx eslint src/` antes da refatoração.

Na leitura manual eu tinha anotado "muitos ifs", mas sem medida nenhuma. A complexidade cognitiva transforma isso em um número (27) e, mais importante, pesa o aninhamento: um `if` dentro de outro `if`

dentro de um `for` custa mais do que três `if`s em sequência, o que bate com a dificuldade real de acompanhar o código. A segunda regra, `no-collapsible-if` (linha 43), apontou o `else if (user.role === 'USER') { if (item.value <= 500) ... }`, um `if` que poderia ser unido ao de fora. Esse eu não tinha notado.

Por outro lado, a ferramenta não encontrou tudo. A duplicação das linhas CSV e HTML não foi reportada, porque `no-identical-functions` compara funções inteiras e aqui as repetições estão dentro de uma única função. Os números mágicos também não aparecem no preset recomendado. As duas análises acabaram se completando.

4. Processo de refatoração

Escolhi como smell mais crítico o método longo com condicionais aninhadas, porque ele é a causa dos outros dois: a duplicação só existe porque cada ramo do `if` precisou da sua própria cópia da formatação.

Antes (corpo do laço em `generateReport`):

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) {
      item.priority = true;
    }
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority ? ' style="font-weight:bold;"' : '';
      report += `<tr${style}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') {
        report += `${item.id},${item.name},${item.value},${user.name}\n`;
        total += item.value;
      } else if (reportType === 'HTML') {
        report += `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
        total += item.value;
      }
    }
  }
}
// + dois blocos if (reportType === ...) iguais a esses no cabeçalho e no rodapé
```

Depois (trechos de `src/ReportGenerator.refactored.js`):

```
const STANDARD_USER_MAX_VALUE = 500;
const PRIORITY_VALUE_THRESHOLD = 1000;

const standardUserPolicy = {
  selectItems: (items) => items.filter((item) => item.value <= STANDARD_USER_MAX_VALUE),
  isHighlighted: () => false,
};
// adminPolicy: devolve todos os itens e marca priority nos que passam de 1000

const csvFormat = {
  header: () => 'ID,NOME,VALOR,USUARIO\n',
  row: (item, user) => `${item.id},${item.name},${item.value},${user.name}\n`,
  footer: (total) => `\nTotal,,\n${total},,\n`,
};
// htmlFormat: mesma interface (header, row, footer)

const ROLE_POLICIES = new Map([['ADMIN', adminPolicy], ['USER', standardUserPolicy]]);
const REPORT_FORMATS = new Map([['CSV', csvFormat], ['HTML', htmlFormat]]);
const sumValues = (items) => items.reduce((total, item) => total + item.value, 0);

generateReport(reportType, user, items) {
  const format = REPORT_FORMATS.get(reportType) ?? unknownFormat;
  const policy = ROLE_POLICIES.get(user.role) ?? noAccessPolicy;
  const visibleItems = policy.selectItems(items);

  const rows = visibleItems.map((item) => format.row(item, user, policy.isHighlighted(item)));
  const report = format.header(user) + rows.join('') + format.footer(sumValues(visibleItems));
```

```
  return report.trim();
}
```

As técnicas aplicadas foram:

- **Replace Conditional with Polymorphism** nos formatos: CSV e HTML viraram objetos com a mesma interface (`header`, `row`, `footer`), escolhidos uma única vez pelo `Map`. Os quatro `if (reportType === ...)` sumiram, e um formato novo agora é um objeto novo mais uma entrada no `Map`.
- A mesma técnica nos **papéis**: cada política tem `selectItems` e `isHighlighted`. A regra de prioridade ficou dentro da política de admin, que é o único lugar onde ela vale.
- **Extract Function**: `sumValues` substituiu os quatro `total +=`, e cada linha passou a ser montada em um único `row` por formato, o que eliminou a duplicação.
- **Replace Magic Number with Symbolic Constant** para 500 e 1000.
- **Introduce Special Case**: `unknownFormat` e `noAccessPolicy` cobrem tipo ou papel desconhecido sem precisar de `if`, mantendo o comportamento original (tipo desconhecido gera string vazia; papel desconhecido gera relatório sem linhas e total 0).

Uma decisão consciente foi manter o `item.priority = true` na política de admin. Alterar os objetos recebidos não é o ideal, mas faz parte do comportamento observável do código original, e refatoração não deve mudar comportamento.

No fim, `generateReport` caiu de cerca de 60 para 8 linhas, sem nenhum `if`, e o ESLint não aponta mais erro no arquivo refatorado. O `src/ReportGenerator.js` original continua no repositório com os 2 erros, para comparação.

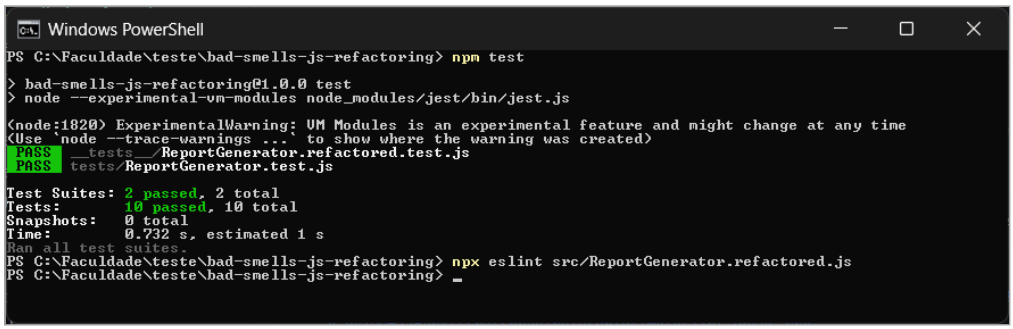


Figura 2: as duas suítes passando (10 testes) e o ESLint sem erros em `ReportGenerator.refactored.js`.

5. Conclusão

O que mais ficou claro neste trabalho é que a rede de segurança só serve se estiver de pé antes de começar. Aqui ela não estava: os testes não rodavam e, quando rodaram, mostraram um defeito real. Se eu tivesse começado a refatorar direto, teria duas opções ruins: copiar o bug para a versão nova ou corrigi-lo sem perceber e depois não saber se a diferença no resultado era intencional. Por isso separei "fazer os testes passarem" de "refatorar", cada coisa no seu commit.

Durante a refatoração rodei os testes a cada passo. Como eles verificam a saída e não a implementação, consegui trocar toda a estrutura interna sem tocar em nenhum teste; a cópia usada para o arquivo novo difere da original apenas na linha do `import`.

Quanto à qualidade, o ganho não está em o código ter ficado mais curto, e sim em ter ficado mais fácil de mudar. Incluir um formato ou um papel agora é acrescentar um objeto, sem mexer no que já funciona, e cada parte pode ser lida e testada sozinha. A complexidade cognitiva saindo de 27 para dentro do limite de 5 é a forma medível dessa melhora.